

Task 9: Session 9 (AIML) - Assignment Questions

Name: Mihir Sopan Patil

Domain: AIML

College & Branch Name: Zeal Polytechnic, Diploma In AIML

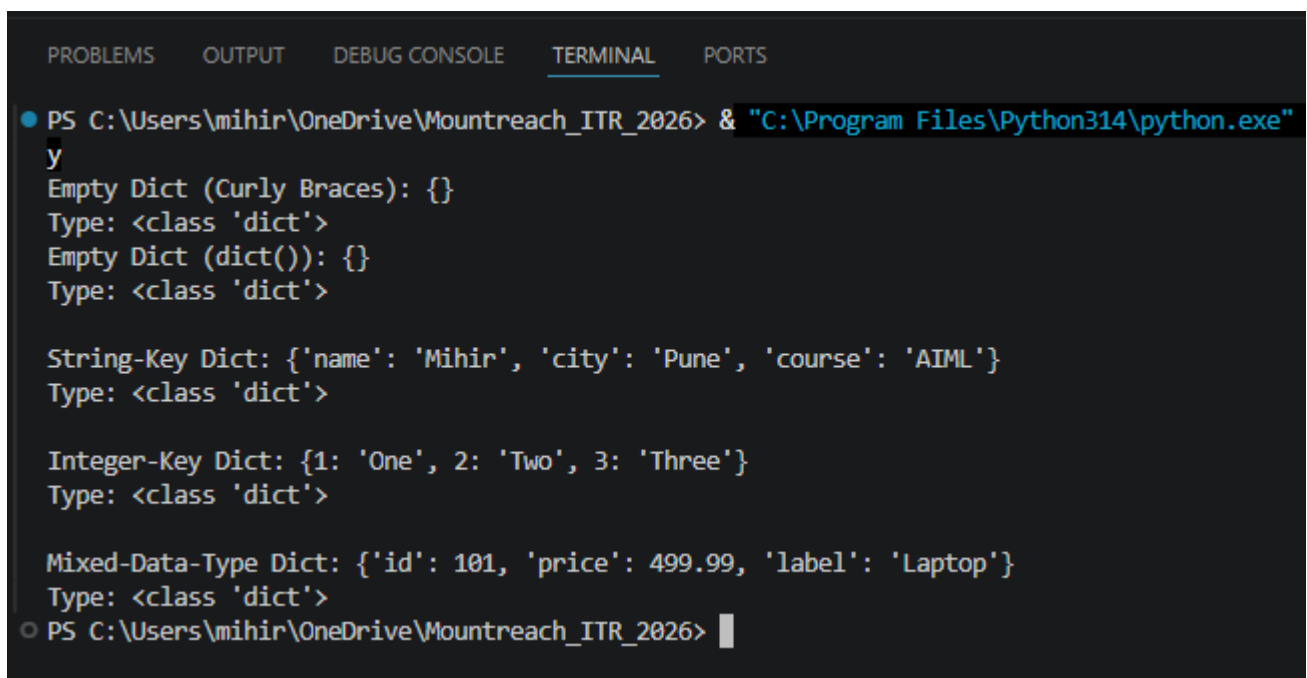
1. Creating Dictionaries

Write a program to create dictionaries using different methods:

- An empty dictionary using two different ways.
- A dictionary with string keys (name, city, course).
- A dictionary with integer keys.
- A mixed data type dictionary (string, int, float).

Print each dictionary and its type. Add comments explaining the syntax.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
y
Empty Dict (Curly Braces): {}
Type: <class 'dict'>
Empty Dict (dict()): {}
Type: <class 'dict'>

String-Key Dict: {'name': 'Mihir', 'city': 'Pune', 'course': 'AIML'}
Type: <class 'dict'>

Integer-Key Dict: {1: 'One', 2: 'Two', 3: 'Three'}
Type: <class 'dict'>

Mixed-Data-Type Dict: {'id': 101, 'price': 499.99, 'label': 'Laptop'}
Type: <class 'dict'>
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
# Empty Dictionary - Two Different Ways
empty_dict_1 = {}           # Method 1: Using Curly Braces
empty_dict_2 = dict()       # Method 2: Using The dict() Constructor
print("Empty Dict (Curly Braces):", empty_dict_1)
print("Type:", type(empty_dict_1))
print("Empty Dict (dict()):", empty_dict_2)
print("Type:", type(empty_dict_2))

# Dictionary With String Keys
string_key_dict = {
    "name": "Mihir",
    "city": "Pune",
    "course": "AIML"
}

print("\nString-Key Dict:", string_key_dict)
print("Type:", type(string_key_dict))

# Dictionary With Integer Keys
int_key_dict = {
    1: "One",
    2: "Two",
    3: "Three"
}

print("\nInteger-Key Dict:", int_key_dict)
print("Type:", type(int_key_dict))

# Mixed Data Type Dictionary (Keys/Values Can Mix str, int, float)
mixed_dict = {
    "id": 101,           # Int Value
    "price": 499.99,     # Float Value
    "label": "Laptop"    # String Value
}

print("\nMixed-Data-Type Dict:", mixed_dict)
print("Type:", type(mixed_dict))
```

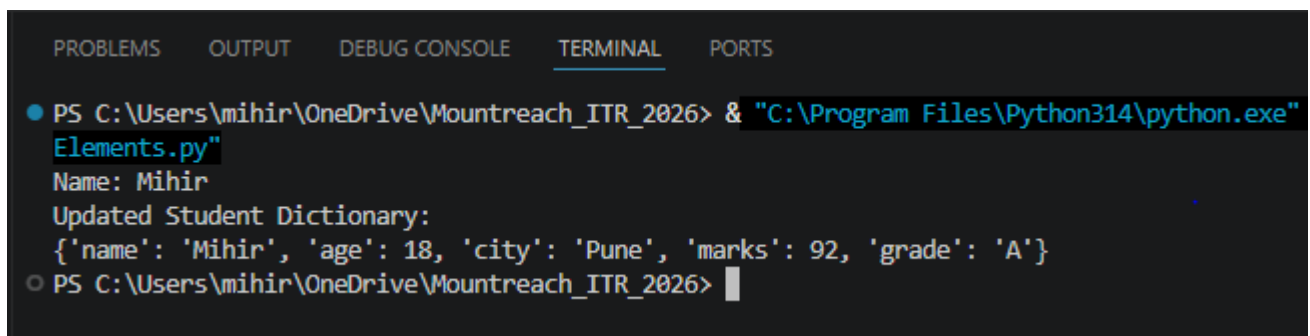
2. Accessing & Modifying Elements

Create the dictionary:

```
student = {  
    "name": "YourName",  
    "age": 00,  
    "city": "YourCity",  
    "marks": 00  
}
```

- Print the value of "name" using square brackets.
- Update the "marks" to 92.
- Add a new key "grade" with value "A".
- Print the updated dictionary.

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"  
  Elements.py  
  Name: Mihir  
  Updated Student Dictionary:  
  {'name': 'Mihir', 'age': 18, 'city': 'Pune', 'marks': 92, 'grade': 'A'}  
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
student = {  
    "name": "Mihir",  
    "age": 18,  
    "city": "Pune",  
    "marks": 84  
}  
  
# Accessing A Value Using Square Brackets  
print("Name:", student["name"])  
  
# Updating An Existing Key's Value  
student["marks"] = 92  
  
# Adding A New Key-Value Pair  
student["grade"] = "A"  
  
print("Updated Student Dictionary:")  
print(student)
```

3. get(), keys(), values(), items()

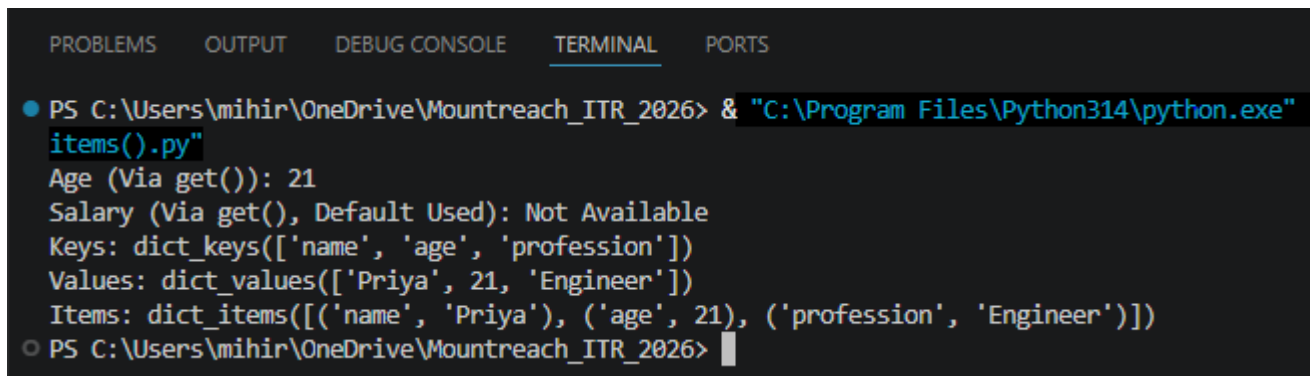
Create a dictionary:

```
person = {"name": "Priya", "age": 21, "profession": "Engineer"}
```

Demonstrate:

- Using get() to access "age" and a non-existing key "salary" (with default value).
- Print all keys using keys().
- Print all values using values().
- Print all key-value pairs using items().

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe" items().py
Age (Via get()): 21
Salary (Via get(), Default Used): Not Available
Keys: dict_keys(['name', 'age', 'profession'])
Values: dict_values(['Priya', 21, 'Engineer'])
Items: dict_items([('name', 'Priya'), ('age', 21), ('profession', 'Engineer')])
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> █
```

Program Code:

```
person = {"name": "Priya", "age": 21, "profession": "Engineer"}

# Safely Access A Key; Second Argument Is The Default If Key Is Missing
print("Age (Via get()):", person.get("age"))
print("Salary (Via get(), Default Used):", person.get("salary", "Not Available"))

# All Keys
print("Keys:", person.keys())

# All Values
print("Values:", person.values())

# All Key-Value Pairs
print("Items:", person.items())
```

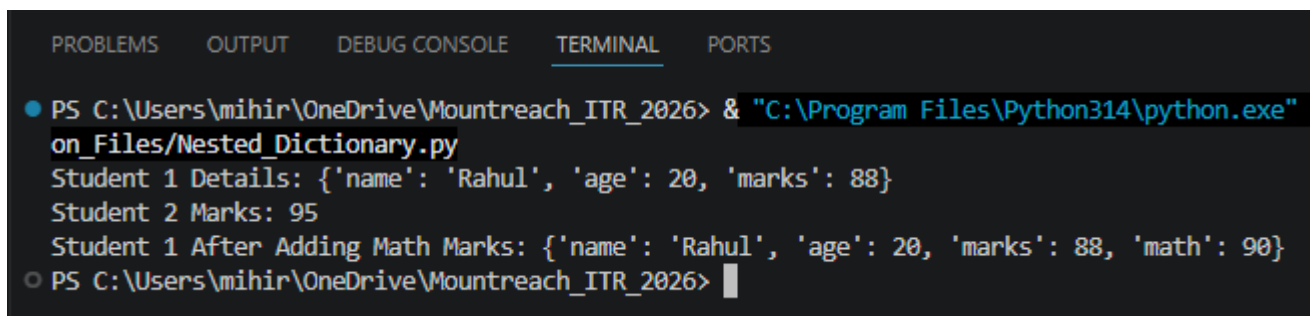
4. Nested Dictionary

Create a nested dictionary to store information of two students:

```
students = {  
    "s1": {"name": "Rahul", "age": 20, "marks": 88},  
    "s2": {"name": "Sneha", "age": 21, "marks": 95}  
}
```

- Print the details of the first student.
- Print the marks of the second student.
- Add a new subject "math" with marks 90 for the first student.

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"  
on_Files/Nested_Dictionary.py  
Student 1 Details: {'name': 'Rahul', 'age': 20, 'marks': 88}  
Student 2 Marks: 95  
Student 1 After Adding Math Marks: {'name': 'Rahul', 'age': 20, 'marks': 88, 'math': 90}  
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> █
```

Program Code:

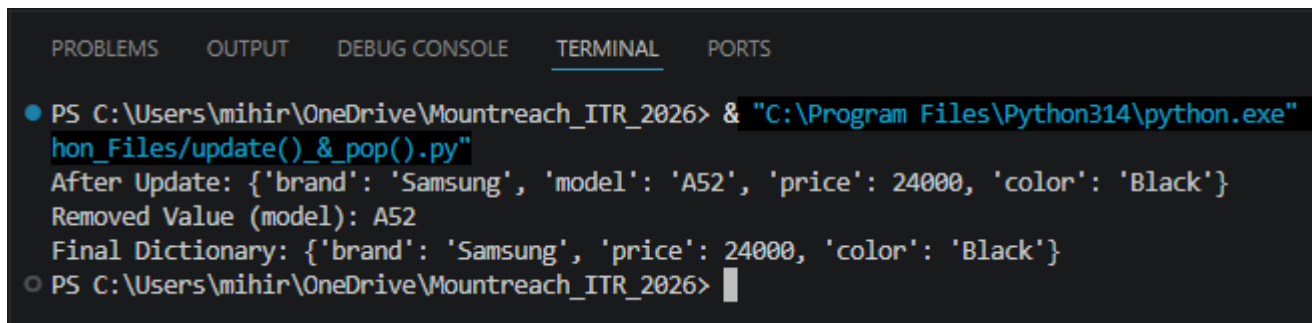
```
students = {  
    "s1": {"name": "Rahul", "age": 20, "marks": 88},  
    "s2": {"name": "Sneha", "age": 21, "marks": 95}  
}  
  
# Details Of The First Student (Entire Inner Dictionary)  
print("Student 1 Details:", students["s1"])  
  
# Marks Of The Second Student (Specific Value From Inner Dictionary)  
print("Student 2 Marks:", students["s2"]["marks"])  
  
# Adding A New Subject For The First Student  
students["s1"]["math"] = 90  
print("Student 1 After Adding Math Marks:", students["s1"])
```

5. update() & pop()

Create a dictionary: info = {"brand": "Samsung", "model": "A52", "price": 25000}

- Update it with new information: {"color": "Black", "price": 24000}.
- Remove the key "model" using pop() and print the removed value.
- Print the final dictionary.

Output:



```
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
hon_Files/update()_&_pop().py"
After Update: {'brand': 'Samsung', 'model': 'A52', 'price': 24000, 'color': 'Black'}
Removed Value (model): A52
Final Dictionary: {'brand': 'Samsung', 'price': 24000, 'color': 'Black'}
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026>
```

Program Code:

```
info = {"brand": "Samsung", "model": "A52", "price": 25000}

# update() Merges/Overwrites Keys From The Given Dictionary
info.update({"color": "Black", "price": 24000})
print("After Update:", info)

# pop() Removes A Key And Returns Its Value
removed_value = info.pop("model")
print("Removed Value (model):", removed_value)

print("Final Dictionary:", info)
```

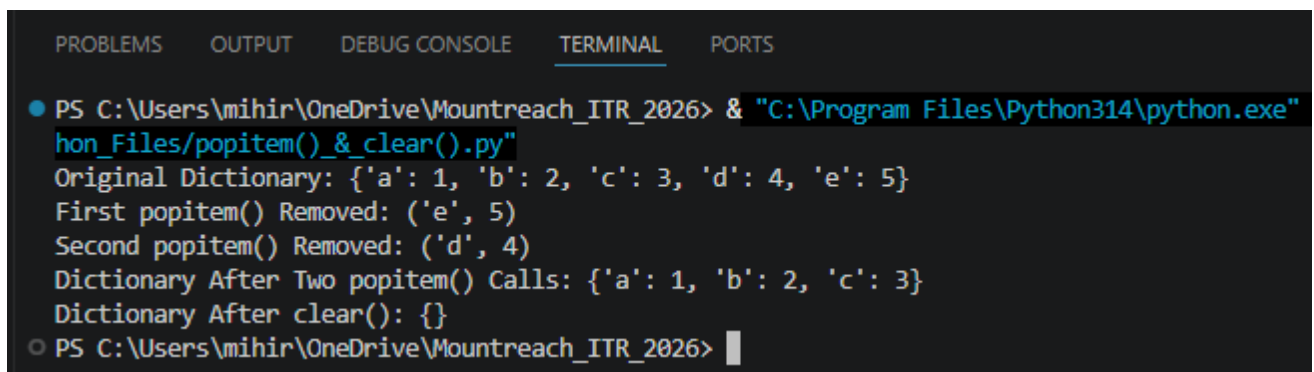
6. popitem() & clear()

Create a dictionary with at least 5 key-value pairs.

- Use popitem() twice and print the removed items each time.
- Clear the entire dictionary using clear().
- Print the final result.

Add comments explaining the difference between pop() and popitem().

Output:



```
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
hon_Files/popitem()_&_clear().py"
Original Dictionary: {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
First popitem() Removed: ('e', 5)
Second popitem() Removed: ('d', 4)
Dictionary After Two popitem() Calls: {'a': 1, 'b': 2, 'c': 3}
Dictionary After clear(): {}
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
sample_dict = {"a": 1, "b": 2, "c": 3, "d": 4, "e": 5}
print("Original Dictionary:", sample_dict)

# popitem() Removes & Returns The Last Inserted (Key, Value) Pair
removed_1 = sample_dict.popitem()
print("First popitem() Removed:", removed_1)

removed_2 = sample_dict.popitem()
print("Second popitem() Removed:", removed_2)

print("Dictionary After Two popitem() Calls:", sample_dict)

# clear() Empties The Dictionary Completely
sample_dict.clear()
print("Dictionary After clear():", sample_dict)

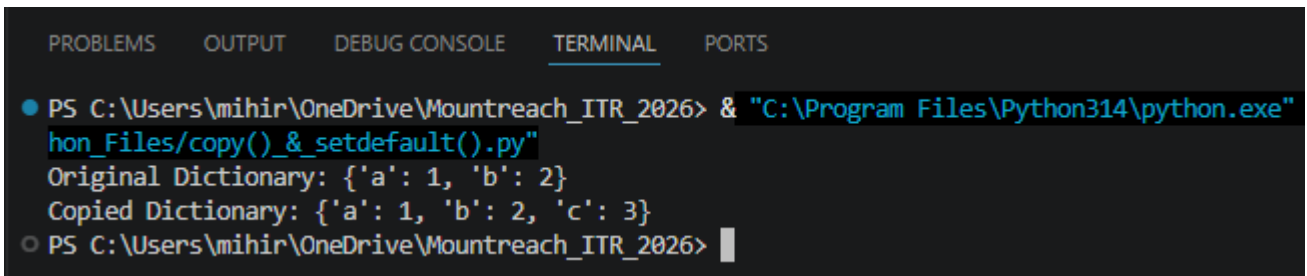
# Difference Between pop() & popitem():
# - pop(key) Removes A Specific Key You Name, & Returns Its Value.
#   It Raises A KeyError If The Key Doesn't Exist (Unless A Default Is Given).
# - popitem() Removes & Returns The Last Inserted (Key, Value) Pair As A Tuple.
#   You Don't Choose Which Item Is Removed; Python Picks The Most Recently Added One.
```

7. copy() & setdefault()

Create a dictionary: `d = {"a": 1, "b": 2}`

- Make a copy of it.
- Use `setdefault()` to add key "c" with value 3 (if it doesn't exist).
- Use `setdefault()` again for an existing key "a".
- Print the original and copied dictionaries.

Output:



```
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe" hon_Files/copy()_&_setdefault().py
Original Dictionary: {'a': 1, 'b': 2}
Copied Dictionary: {'a': 1, 'b': 2, 'c': 3}
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026>
```

Program Code:

```
d = {"a": 1, "b": 2}

# copy() Creates A Separate (Shallow) Copy; Changes To One Don't Affect The Other
d_copy = d.copy()

# setdefault() Adds The Key With The Given Value ONLY If It Doesn't Already Exist
d_copy.setdefault("c", 3)          # "c" Doesn't Exist -> Gets Added With Value 3
d_copy.setdefault("a", 100)        # "a" Already Exists -> Value Stays Unchanged

print("Original Dictionary:", d)
print("Copied Dictionary:", d_copy)
```


8. fromkeys() & Membership

Write a program that:

- Uses dict.fromkeys() to create a dictionary with keys ["name", "age", "city"] and default value None.
- Takes user input to fill these keys.
- Checks if a given key exists using in operator.

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
  hon_Files/fromkeys()_&_membership.py"
Initial Dictionary: {'name': None, 'age': None, 'city': None}
Enter Your name:Mihir
Enter Your age:18
Enter Your city:Pune
Filled Dictionary: {'name': 'Mihir', 'age': '18', 'city': 'Pune'}
Enter A Key To Check If It Exists: age
age Exists In The Dictionary.
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
# Create A Dictionary With Given Keys, All Set To None Initially
user_info = dict.fromkeys(["name", "age", "city"], None)
print("Initial Dictionary:", user_info)

# Take User Input To Fill In The Keys
for key in user_info:
    user_info[key] = input("Enter Your " + key + ":")

print("Filled Dictionary:", user_info)

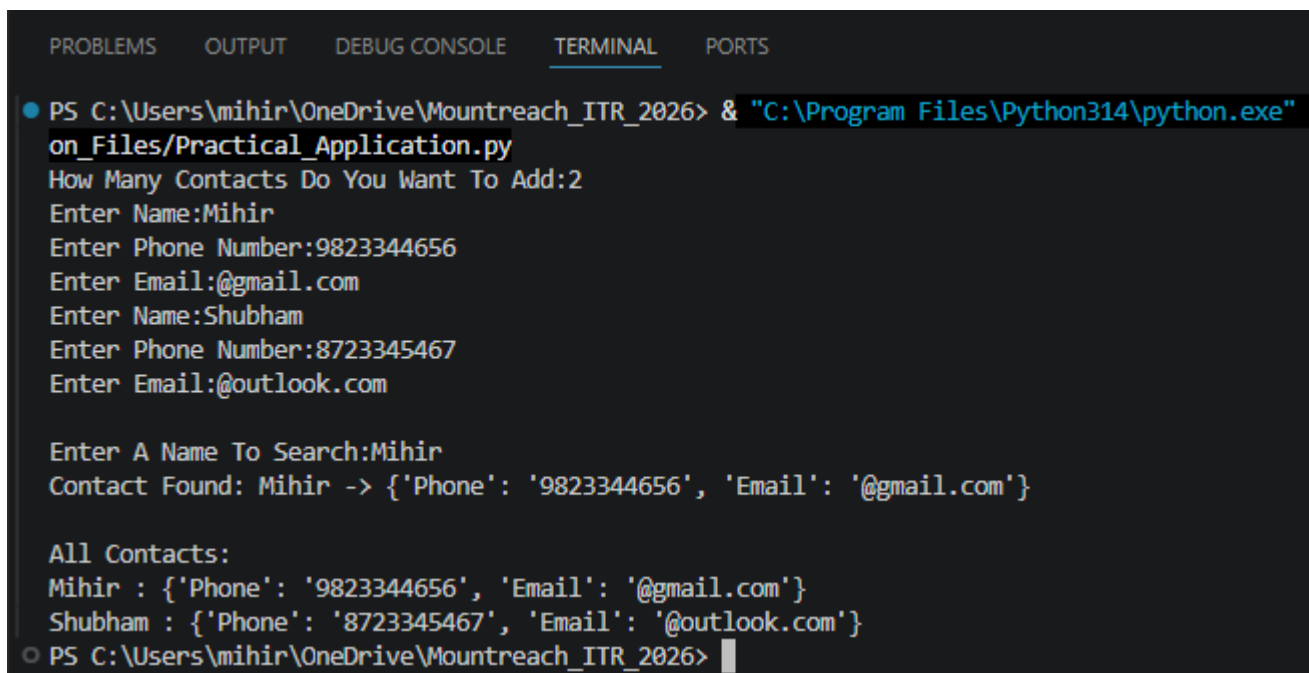
# Membership Check Using The 'in' Operator
check_key = input("Enter A Key To Check If It Exists: ")
if check_key in user_info:
    print(check_key, "Exists In The Dictionary.")
else:
    print(check_key, "Does Not Exist In The Dictionary.")
```

9. Practical Application

Write a program to store and manage contact information using a dictionary:

- Take name, phone number, and email as input.
- Store them in a dictionary with name as key.
- Allow the user to search a contact by name using `get()`.
- Print all contacts using `items()`.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
on_Files/Practical_Application.py
How Many Contacts Do You Want To Add:2
Enter Name:Mihir
Enter Phone Number:9823344656
Enter Email:@gmail.com
Enter Name:Shubham
Enter Phone Number:8723345467
Enter Email:@outlook.com

Enter A Name To Search:Mihir
Contact Found: Mihir -> {'Phone': '9823344656', 'Email': '@gmail.com'}

All Contacts:
Mihir : {'Phone': '9823344656', 'Email': '@gmail.com'}
Shubham : {'Phone': '8723345467', 'Email': '@outlook.com'}
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> █
```

Program Code:

```
contacts = {}

num_contacts = int(input("How Many Contacts Do You Want To Add:"))
for _ in range(num_contacts):
    name = input("Enter Name:")
    phone = input("Enter Phone Number:")
    email = input("Enter Email:")

    # Store Each Contact's Details As A Nested Dictionary, Keyed By Name
    contacts[name] = {"Phone": phone, "Email": email}

# Search A Contact By Name Using get() (Returns None If Not Found)
search_name = input("\nEnter A Name To Search:")
found_contact = contacts.get(search_name)
```

```
if found_contact:
    print("Contact Found:", search_name, "->", found_contact)
else:
    print("No Contact Found With The Name", search_name,)

# Print All Contacts Using items()
print("\nAll Contacts:")
for name, details in contacts.items():
    print(name, ":", details)
```

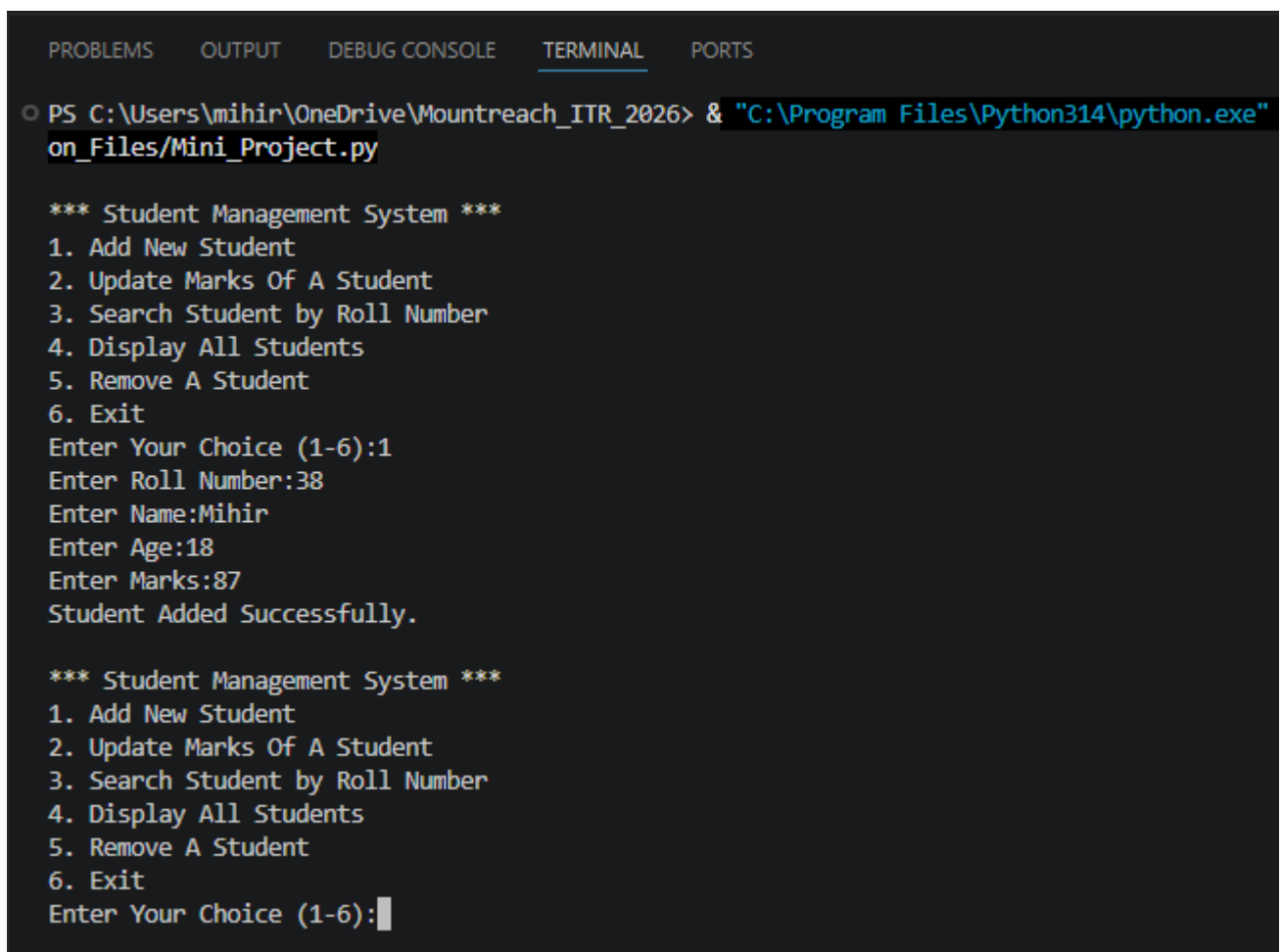
10. Mini Project - Combined Concepts

Create a **Student Management System** using a dictionary:

- Repeatedly ask the user to enter student details (roll number as key, and a nested dictionary containing name, age, and marks).
- Use a while loop with a menu:
 - a) Add new student
 - b) Update marks of a student
 - c) Search student by roll number
 - d) Display all students
 - e) Remove a student
 - f) Exit

Use appropriate dictionary methods (get(), update(), pop(), etc.). Add meaningful comments

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe" on_Files/Mini_Project.py

*** Student Management System ***
1. Add New Student
2. Update Marks Of A Student
3. Search Student by Roll Number
4. Display All Students
5. Remove A Student
6. Exit
Enter Your Choice (1-6):1
Enter Roll Number:38
Enter Name:Mihir
Enter Age:18
Enter Marks:87
Student Added Successfully.

*** Student Management System ***
1. Add New Student
2. Update Marks Of A Student
3. Search Student by Roll Number
4. Display All Students
5. Remove A Student
6. Exit
Enter Your Choice (1-6):
```

Program Code:

```
students_db = {} # roll (key) -> {"name", "age", "marks"} (Nested Dictionary)
```

```
def add_student():
    roll = input("Enter Roll Number:")
    if roll in students_db:
        print("A Student With This Roll Number Already Exists.")
        return
    name = input("Enter Name:")
    age = input("Enter Age:")
    marks = input("Enter Marks:")
    students_db[roll] = {"name": name, "age": age, "marks": marks}
    print("Student Added Successfully.")
```

```
def update_marks():
    roll = input("Enter Roll Number To Update Marks:")
    if roll in students_db:
        new_marks = input("Enter New Marks:")
        students_db[roll]["marks"] = new_marks
        print("Marks Updated Successfully.")
    else:
        print("Student Not Found.")
```

```
def search_student():
    roll = input("Enter Roll Number To Search:")
    # get() Avoids A KeyError If The Roll Number Doesn't Exist
    student_record = students_db.get(roll)
    if student_record:
        print("Student Found -> Roll Number", roll, ":", student_record)
    else:
        print("Student Not Found.")
```

```
def display_all():
    if not students_db:
        print("No Students In The System Yet.")
        return
    for roll, details in students_db.items():
        print("Roll Number", roll, ":", details)
```

```
def remove_student():
    roll = input("Enter Roll Number To Remove: ")
    removed = students_db.pop(roll, None) # Default None Avoids KeyError
    if removed:
```

```

        print("Removed Student:", removed)
    else:
        print("Student Not Found.")

def show_menu():
    print("\n*** Student Management System ***")
    print("1. Add New Student")
    print("2. Update Marks Of A Student")
    print("3. Search Student by Roll Number")
    print("4. Display All Students")
    print("5. Remove A Student")
    print("6. Exit")

while True:
    show_menu()
    choice = input("Enter Your Choice (1-6):")

    if choice == "1":
        add_student()
    elif choice == "2":
        update_marks()
    elif choice == "3":
        search_student()
    elif choice == "4":
        display_all()
    elif choice == "5":
        remove_student()
    elif choice == "6":
        print("Exiting Student Management System. Goodbye!")
        break
    else:
        print("Invalid Choice. Please Enter A Number Between 1 & 6.")

```